

PEGELCONNECT PRO

Prozessorientierter Projektbericht (POB)

PegelConnect Pro

Projekt	PegelConnect Pro
Autor	Bünyamin Atik
Schwerpunkt	Fachinformatik Systemintegration + Anwendungsentwicklung
Projektart	Eigenständiges Lern-, Integrations- und Referenzprojekt
Technologien	Java 17, MQTT/Mosquitto, REST, NGINX, systemd, Vagrant, Ubuntu 24.04
Stand	August 2026

Ziel des Berichts: Nachvollziehbare Darstellung des Projektprozesses von der Ausgangslage über Planung, Umsetzung und Fehlerbehebung bis zu Test, Abnahme und Reflexion.

Inhaltsverzeichnis

1. 1. Projektkontext und Ausgangslage
2. 2. Zieldefinition und Abgrenzung
3. 3. Projektplanung und Vorgehensmodell
4. 4. Technische Konzeption und Architekturentscheidungen
5. 5. Umsetzung - FIAE-Anteil
6. 6. Umsetzung - FISI-Anteil
7. 7. Fehleranalyse und prozessorientierte Korrekturen
8. 8. Test, Abnahme und Qualitätssicherung
9. 9. Ergebnis und Soll-Ist-Vergleich
10. 10. Reflexion und Lerngewinn
11. 11. Ausblick
12. 12. Anlagen- und Quellenhinweise

1. Projektkontext und Ausgangslage

PegelConnect Pro ist ein eigenständiges Lern- und Referenzprojekt, das die beiden Perspektiven Anwendungsentwicklung (FIAE) und Systemintegration (FISI) in einem durchgängigen technischen System zusammenführt. Das Projekt baut auf der fachlichen Idee einer Pegelüberwachung auf, wird jedoch als eigene Pro-Variante mit erweitertem Funktionsumfang umgesetzt.

Ausgangslage war eine funktionsfähige Grundidee mit Wasserstandsdaten, MQTT und Visualisierung. Für die Pro-Variante sollte daraus ein reproduzierbares, dokumentiertes und testbares Gesamtsystem entstehen, das nicht nur eine Oberfläche zeigt, sondern auch Betrieb, Konfiguration, Fehlerverhalten und Wiederanlauf nachvollziehbar abbildet.

1.1 Fachlicher Bedarf

- Aktuelle Wasserstände für KÖLN, BONN und MAINZ abrufen und darstellen.
- Messwerte über MQTT bereitstellen und den Verbindungsstatus signalisieren.
- Zusätzliche Wetterdaten und historische Pegelverläufe integrieren.
- Ein eigenes Web-Dashboard mit Karten-, Report- und Exportfunktionen bereitstellen.
- Die komplette Umgebung reproduzierbar in einer Linux-VM aufsetzen.
- Fehlerfälle nicht nur theoretisch beschreiben, sondern praktisch testen und dokumentieren.

1.2 Technischer Ist-Zustand zu Projektbeginn

Zu Beginn waren mehrere Komponenten bereits als Einzelbausteine vorhanden oder in Vorstufen erprobt. Es fehlte jedoch eine konsistente End-to-End-Struktur mit externer Konfiguration, stabiler Service-Kopplung, Reverse Proxy, systematischem Testbericht und vollständiger Dokumentation.

2. Zieldefinition und Abgrenzung

2.1 Soll-Zustand

Der Soll-Zustand war ein vollständig startbares PegelConnect-Pro-System mit Java-Backend, realen externen APIs, MQTT-Broker, eigenem Web-Dashboard, NGINX-Reverse-Proxy und automatisierter VM-Bereitstellung.

- Java 17 Backend mit Maven-Build.
- PEGELONLINE als Wasserstandsdatenquelle.
- Open-Meteo als zusätzliche Wetterdatenquelle.
- Mosquitto als MQTT-Broker mit QoS 1, Retain und Last Will.
- REST-Endpunkte für Zustand, Historie, Wetter und Laufzeitkonfiguration.
- NGINX als zentraler Browser-Einstiegspunkt.
- systemd für Autostart und Recovery.
- Vagrant/VirtualBox für reproduzierbare Bereitstellung.
- Historie für 24h, 7d und 30d sowie CSV-, JSON- und PDF-Reports.
- Technische Dokumentation, Testbericht und Schreibtischtest im GitHub-Repository.

2.2 Abgrenzung

PegelConnect Pro ist bewusst eine separate Erweiterung. Es verändert weder das ursprüngliche FISI- noch das FIAE-Team-Repository. Historische Daten, Wetter, eigene Weboberfläche, Karten und Reports gehören zur Pro-Erweiterung.

3. Projektplanung und Vorgehensmodell

Der Projektprozess wurde iterativ und risikoorientiert durchgeführt. Nach jeder größeren Änderung wurde zuerst kompiliert, anschließend provisioniert und danach im laufenden System getestet. Erst nach einem stabilen Zwischenstand wurde der nächste Funktionsblock umgesetzt.

3.1 Prozessphasen

Phase	Inhalt
1. Analyse	Anforderungen und vorhandene Komponenten prüfen; Trennung FISI/FIAE definieren.
2. Konzeption	Architektur, Ports, Konfigurationspfade, MQTT- und REST-Datenfluss festlegen.
3. Implementierung	Java-Clients, Konfigurationslogik, REST-Endpunkte, Dashboard und Infrastruktur ergänzen.
4. Integration	Vagrant, systemd, Mosquitto und NGINX zu einem End-to-End-System verbinden.
5. Test	Happy-Path-, Fehler-, Restart-, Last-Will- und Recovery-Tests ausführen.
6. Korrektur	Gefundene Fehler reproduzieren, Ursache eingrenzen und nachhaltig im Provisioning beheben.
7. Abnahme	Alle Dienste, APIs, Historien, MQTT und Recovery erneut prüfen.
8. Dokumentation	Technische Doku, Testbericht, Schreibtischtest und POB im Repository bündeln.

3.2 Qualitätsprinzip

Wesentlich war die Reihenfolge „Ändern -> Kompilieren -> Provisionieren -> Laufzeittest -> Fehleranalyse -> Retest“. Dadurch wurden Fehler nicht verdeckt, sondern als Teil des Projektprozesses sichtbar und dokumentierbar.

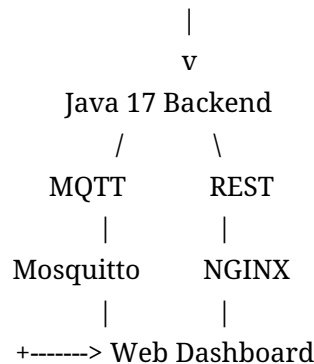
4. Technische Konzeption und Architekturentscheidungen

4.1 Gesamtarchitektur

Die Architektur trennt Datenquellen, Verarbeitung, Messaging, Webzugriff und Betrieb. Das Java-Backend ruft PEGELONLINE und Open-Meteo ab, verwaltet Messwerte, publiziert Wasserstände

per MQTT und stellt REST-Endpunkte bereit. NGINX veröffentlicht Dashboard und API nach außen.

PEGELONLINE + Open-Meteo + config.json



4.2 Port- und Dienstkonzept

Komponente	Gast-Port	Host-Port	Rolle
Java Backend	8080	8080	REST und statische Ressourcen
Mosquitto	1883	1885	MQTT Broker
NGINX	80	8088	Reverse Proxy / Browser-Einstieg

4.3 Externe Konfiguration

Eine zentrale Architekturentscheidung war die Entfernung von Hardcoding. Die Anwendung lädt Konfiguration in folgender Priorität:

13. Pfad aus PEGELCONNECT_CONFIG
14. /etc/pegelconnect-pro/config.json
15. lokale config/config.json
16. interne Defaultwerte

Dadurch können Broker, Client-ID, Stationen, Intervalle, API-Server, Ports und weitere Parameter ohne Quellcodeänderung angepasst werden.

5. Umsetzung - FIAE-Anteil

5.1 Zentrale Softwarekomponenten

Klasse	Aufgabe
PegelConnectPro	Startpunkt; initialisiert Konfiguration, Clients, MQTT, Webserver und Scheduler.
AppConfig	Lädt und validiert die externe Laufzeitkonfiguration.
PegelOnlineClient	Kapselt aktuelle und historische PEGELONLINE-REST-Aufrufe.

WeatherClient	Ruft Wetterdaten anhand der Stationskoordinaten ab.
StationReading	Datenmodell mit Station, timestamp, value, unit und trend.
MqttGateway	Verbindungsaufbau, Last Will, Publish, Status und Reconnect.
WebServer	REST-Endpunkte und statische Webressourcen.

5.2 Datenverarbeitung

17. Konfiguration laden.
18. Aktuellen Pegelwert je Station abrufen.
19. API-Antwort in internes Datenmodell überführen.
20. Trend gegen den vorherigen gültigen Wert bestimmen.
21. Aktuellen Zustand im ReadingStore speichern.
22. Messwert über MQTT veröffentlichen.
23. Zustand und Historie über REST bereitstellen.

5.3 Fehlerbehandlung

Ungültige Stationen und Zeiträume werden kontrolliert mit HTTP 400 beantwortet. Externe API-Fehler sollen geloggt werden, ohne ungültige Werte zu publizieren. Dadurch wird zwischen fachlichen Fehlern und einem kompletten Systemausfall unterschieden.

6. Umsetzung - FISI-Anteil

6.1 VM und Provisioning

- Ubuntu 24.04 als Gastbetriebssystem.
- 2 vCPU und 2048 MB RAM.
- Vagrant und VirtualBox für automatisiertes Aufsetzen.
- Shared Folder bewusst deaktiviert; Dateien werden explizit provisioniert.
- Java 17, Maven, Mosquitto, NGINX und curl werden automatisch installiert.

6.2 systemd

Das Java-Backend wird als pegelconnect-pro.service betrieben. Mosquitto und NGINX laufen ebenfalls als systemd-Dienste. Alle drei Dienste sind für Autostart aktiviert.

6.3 NGINX

NGINX ist der zentrale Webzugang. Der Reverse Proxy leitet Browser- und API-Anfragen an das Java-Backend weiter. Ein gestopptes Backend führt erwartungsgemäß zu HTTP 502; nach dem Start ist die API wieder mit HTTP 200 erreichbar.

6.4 MQTT

Mosquitto übernimmt die MQTT-Vermittlung. Wasserstände werden auf Stations-Topics veröffentlicht; pegel/status signalisiert online/offline. Last Will und Reconnect wurden praktisch getestet.

7. Fehleranalyse und prozessorientierte Korrekturen

Die wichtigsten Lern- und Qualitätsgewinne entstanden durch reale Fehlerfälle. Zwei Probleme wurden im Projektprozess identifiziert und dauerhaft korrigiert.

7.1 Doppelter Mosquitto Listener

Während eines Provisioning-Laufs startete mosquitto.service nicht mehr. Die Analyse mit systemctl status und grep über /etc/mosquitto zeigte zwei Konfigurationsdateien, die beide listener 1883 definierten.

Ursache: pegelconnect-pro.conf und pegelconnect.conf waren gleichzeitig aktiv. Durch Entfernen der veralteten Konfiguration wurde der Broker wieder startfähig. Der Fehler wurde anschließend in das Provisioning und Troubleshooting aufgenommen.

7.2 Harte systemd-Abhängigkeit

Im ersten Broker-Ausfalltest führte sudo systemctl stop mosquitto dazu, dass auch pegelconnect-pro fehlschlug. Damit konnte die Java-Anwendung keine eigene Reconnect-Logik mehr ausführen.

Ursache war die harte Kopplung Requires=mosquitto.service. Diese wurde durch Wants=network-online.target mosquitto.service ersetzt. Danach blieb das Backend aktiv, meldete mqttConnected=false und stellte nach Broker-Neustart automatisch mqttConnected=true wieder her.

7.3 Prozessbewertung

Der entscheidende Punkt war nicht nur die Korrektur, sondern die Rückführung der Erkenntnis in die Infrastrukturdefinition. Damit ist die Lösung reproduzierbar und nicht nur manuell in einer einzelnen VM repariert.

8. Test, Abnahme und Qualitätssicherung

Die technische Abnahme bestand aus 14 dokumentierten Testfällen. Nach der Korrektur des T14-Befunds wurden alle Tests erfolgreich abgeschlossen.

ID	Testfall	Ergebnis
T01	Webserver und Backend	PASS
T02	MQTT Broker und Topics	PASS
T03	Weather API für drei Stationen	PASS
T04	History API 24h	PASS mit Hinweis

T05	History API 7d - 672 Messpunkte	PASS
T06	History API 30d - 2880 Messpunkte	PASS
T07	Runtime Configuration	PASS
T08	Ungültige Station	PASS
T09	Ungültige History-Periode	PASS
T10	Autostart und Dienststatus	PASS
T11	VM Neustart / Wiederanlauf	PASS
T12	MQTT Last Will / Prozessabbruch	PASS
T13	NGINX bei Backend-Ausfall	PASS
T14	MQTT Broker-Ausfall / Reconnect	PASS nach Korrektur

8.1 Besondere Testbefunde

- 24h-Historie: 96 Messpunkte; 7d: 672; 30d: 2880.
- Einzelne auffällige historische Pegelwerte wurden als Plausibilitäts-Hinweis dokumentiert.
- Bei 60-Sekunden-Abruf werden identische PEGELONLINE-Zeitstempel mehrfach in die In-Memory-History übernommen; als Verbesserung ist Deduplication nach station + timestamp vorgesehen.
- Last-Will-Test: online -> offline -> online nach hartem Java-Prozessabbruch.
- NGINX-Test: Backend stop -> 502; Backend start -> 200.

8.2 Abnahmeurteil

BESTANDEN. Das System zeigt reproduzierbares Start-, Fehler- und Recovery-Verhalten. Die zentrale Infrastruktur ist automatisiert und die wesentlichen Anwendungsschnittstellen sind getestet.

9. Ergebnis und Soll-Ist-Vergleich

Soll	Ist
Reale Pegeldata für 3 Stationen	Erreicht
Wetterdaten	Erreicht
MQTT + Status + Last Will	Erreicht
REST API	Erreicht
24h/7d/30d Historie	Erreicht
Dashboard + Karte + Verlauf	Erreicht
CSV/JSON/PDF Reports	Erreicht
Externe Konfiguration	Erreicht
NGINX Reverse Proxy	Erreicht
systemd Autostart und Recovery	Erreicht
Vagrant Provisioning	Erreicht
Testbericht + Schreibtischtest + Doku	Erreicht

Der definierte Soll-Zustand wurde erreicht. Darüber hinaus wurden Fehler- und Wiederanlaufverhalten praktisch nachgewiesen, was den Projektwert insbesondere aus FISl-Sicht erhöht.

10. Reflexion und Lerngewinn

10.1 FIAE-Perspektive

- REST-Clients und JSON-Datenverarbeitung praktisch verbinden.
- Datenmodell, Zustandsverwaltung und Trendlogik nachvollziehen.
- Konfiguration aus dem Quellcode herauslösen.
- Fehlerzustände als kontrollierte API-Antworten behandeln.
- MQTT als Messaging-Schnittstelle aus einer Java-Anwendung nutzen.

10.2 FISl-Perspektive

- Linux-Dienste und Abhängigkeiten mit systemd modellieren.
- Reverse Proxy und Portweiterleitung verstehen.
- Provisioning reproduzierbar gestalten.
- Logs und Servicezustände zielgerichtet zur Fehleranalyse nutzen.
- Recovery nicht nur konfigurieren, sondern unter echten Ausfällen testen.

10.3 Persönliche Bewertung

Der größte Lerngewinn lag in der Verbindung beider Fachrichtungen. Ein funktionierender Java-Endpunkt allein reicht nicht aus, wenn der Dienst nicht stabil betrieben wird. Umgekehrt ist eine saubere VM-Infrastruktur ohne zuverlässige Anwendungslogik ebenfalls unvollständig. PegelConnect Pro zeigt deshalb bewusst beide Seiten in einem Projekt.

11. Ausblick

- Deduplication der In-Memory-History nach Station und Zeitstempel.
- Automatisierung der manuellen Testfälle als PowerShell-/Shell-Test-Suite.
- Optional persistente Datenhaltung statt ausschließlich externer Historienabfrage.
- Erweiterte Security-Härtung, falls das System außerhalb einer lokalen Lernumgebung betrieben wird.
- Optional Node-RED als zusätzlicher Visualisierungs-Client, nicht als zwingende Kernkomponente.

Der aktuelle Stand ist für Lern-, Demonstrations- und Portfoliozwecke vollständig nutzbar und dokumentiert.

12. Anlagen- und Quellenhinweise

Dieser POB basiert auf dem tatsächlich implementierten und getesteten Stand des GitHub-Projekts PegelConnect Pro. Verwendete interne Projektdokumente:

- README.md
- docs/01_Projektuebersicht.md
- docs/02_Architektur.md
- docs/03_FIAE_Anwendung.md
- docs/04_FISI_Infrastruktur.md
- docs/05_Konfiguration.md
- docs/06_MQTT_und_Datenfluss.md
- docs/07_REST_API.md
- docs/08_Vagrant_Systemd_NGINX.md
- docs/09_Testbericht.md
- docs/10_Schreibtischtest.md
- docs/11_Troubleshooting.md
- docs/12_Erweiterungen.md
- docs/13_Betrieb_und_Abnahme.md

Repository: <https://github.com/haktel/PegelConnect-Pro>